

# CSE340: Computer Architecture

## Handout\_Chapter - 3: Arithmetic for Computers



I n s p i r i n g   E x c e l l e n c e

Prepared by: Partha Bhoumik (PBK)  
Course Coordinator, CSE340

# Integer Addition-Subtraction

# Addition:  $(7)_{10} + (6)_{10}$

$$\begin{array}{r}
 \phantom{+} \begin{array}{cccccc} & 1 & & 1 & & 0 \end{array} \\
 \phantom{+} \begin{array}{cccccc} 0 & 1 & 1 & 1 & 0 & 1 \end{array} \\
 + \begin{array}{cccccc} 0 & 1 & 1 & 1 & 0 & 0 \end{array} \\
 \hline
 \begin{array}{cccccc} 1 & (1) & 1 & (1) & 0 & (0) \end{array} \phantom{1}
 \end{array}$$

# Bits with  
this color  
represent  
carry-forward

# adding bits right to left.

# Subtraction:  $(7)_{10} - (6)_{10} = (7)_{10} + (-6)_{10}$

$$\begin{array}{r}
 6 = 110 \\
 + 6 = 0110 \\
 = 1001 \\
 + 1 \\
 \hline
 -6 = (1010)_{20}
 \end{array}$$

$$\begin{array}{r}
 \phantom{+} \begin{array}{cccccc} & 1 & & 1 & & 1 & & 0 \end{array} \\
 \phantom{+} \begin{array}{cccccc} 0 & 1 & 1 & 1 & 0 & 1 \end{array} \\
 - \begin{array}{cccccc} 1 & 0 & 1 & 0 & 1 & 0 \end{array} \\
 \hline
 \begin{array}{cccccc} 1 & (1) & 0 & (1) & 0 & (1) & 0 & (0) \end{array} \phantom{1}
 \end{array}$$

# Overflow Detection (integer)

## #Addition:

Case-1: Add two same signed numbers:

if (answer also has same sign):

No overflow

else:

Overflow

Case-2: Add two different signed numbers:

Never Overflow

## #Subtraction:

Case-3: Sub two same signed numbers:

Never Overflow

Case-4: Sub two different signed numbers:

$+A - (-B) = +A + B \Rightarrow$  Case-1

$-A - (+B) = -A + (-B) \Rightarrow$  Case-1

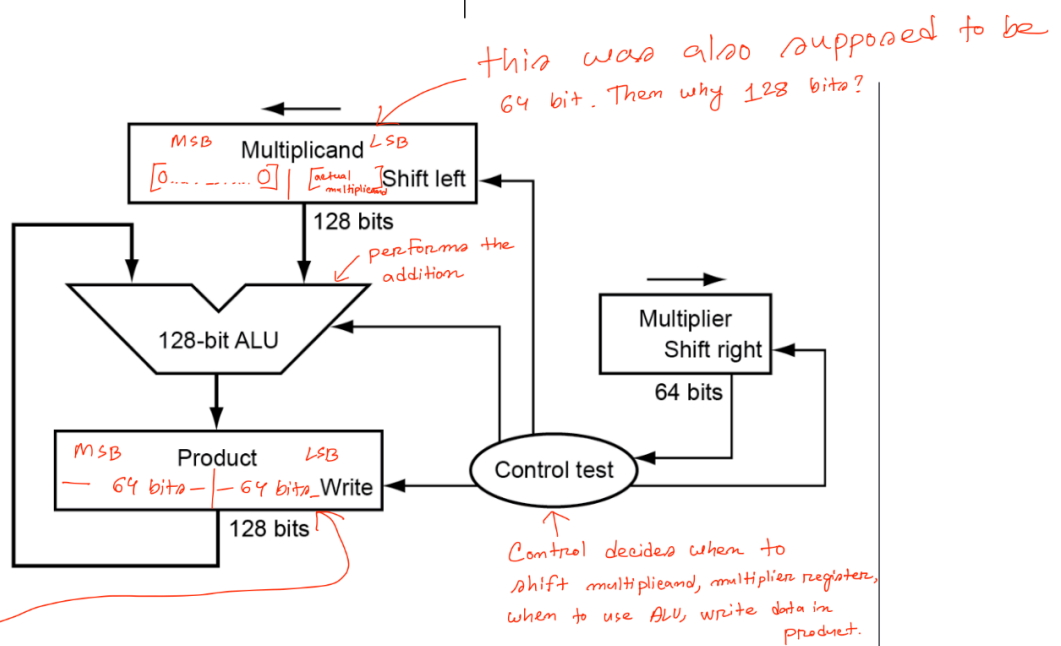
# Long Multiplication

$$\begin{array}{r} 1000 \rightarrow \text{Multiplicand} \\ \times 1001 \rightarrow \text{Multiplier} \\ \hline 1000 \\ 0000 \\ 0000 \\ 1000 \\ \hline 1001000 \rightarrow \text{Product} \end{array}$$

Maximum,

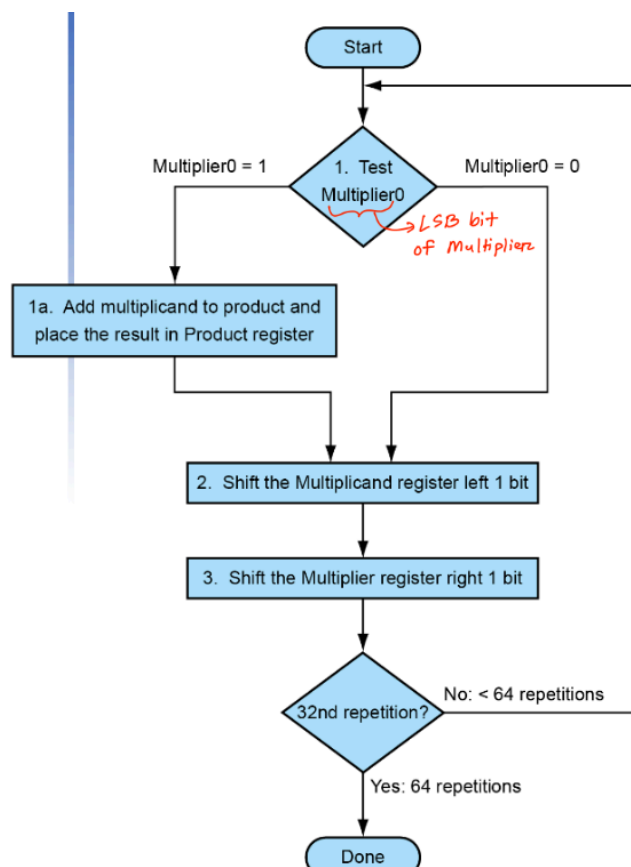
length of the product  
= (length of multiplicand  
+ " " " multiplier)

System:



# Multiply two 64 bit values, product length can be  $(64+64)=128$  bit.  
 But we do not have any 128 bit registers in RISC-V  
 Hence, we use two registers to store the values

Flowchart:



## Example:

Multiply 8 and 9 using the long multiplication method:

Solution:

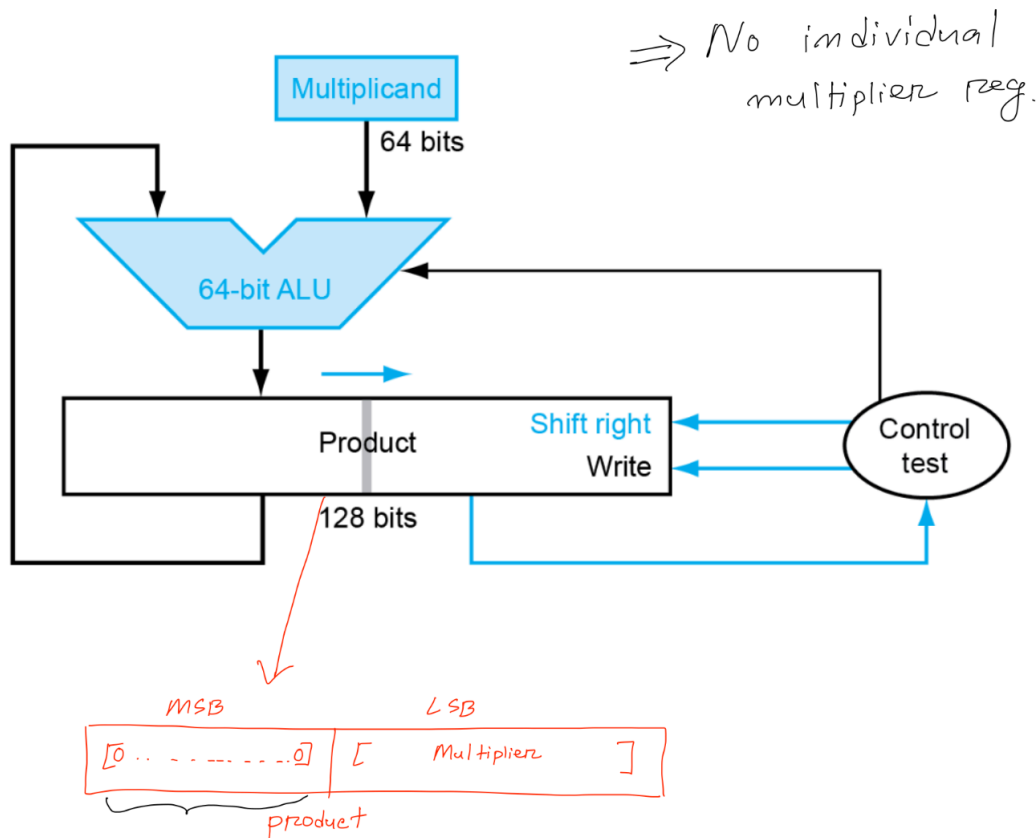
You can choose any of the operands as multiplier or multiplicand.

Number of iterations = Number of bits in Multiplier

| Iteration | Multiplier | Multiplicand | Product   |
|-----------|------------|--------------|-----------|
| 0         | 1001       | 0000 1000    | 0000 0000 |
| 1         | 1001       | 0000 1000    | 0000 1000 |
|           | 1001       | 0001 0000    | 0000 1000 |
|           | 0100       | 0001 0000    | 0000 1000 |
| 2         | 0100       | 0010 0000    | 0000 1000 |
|           | 0010       | 0010 0000    | 0000 1000 |
| 3         | 0010       | 0100 0000    | 0000 1000 |
|           | 0001       | 0100 0000    | 0000 1000 |
| 4         | 0001       | 0100 0000    | 0100 1000 |
|           | 0001       | 1000 0000    | 0100 1000 |
|           | 0000       | 1000 0000    | 0100 1000 |

# Optimized Multiplication

System:



# Number of iteration = Number of bits in Multiplier

Logic:

```

if (iteration <= multiplier bit length):
    if (multiplier0 == 1):
        product_MSB = Multiplicand + product_MSB
        product = right shift product by 1
    elif (multiplier0 == 0):
        product = right shift product by 1
  
```

## Example:

Multiply 8 and 9 using the optimized multiplication method:

Solution:

You can choose any of the operands as multiplier or multiplicand.

Number of iterations = Number of bits in Multiplier

# This color represents the product MSB part.

| Iteration | Multiplicand | Product   |
|-----------|--------------|-----------|
| 0         | 1000         | 0000 1001 |
| 1         | 1000         | 1000 1001 |
|           |              | 0100 0100 |
| 2         | 1000         | 0010 0010 |
| 3         | 1000         | 0001 0001 |
| 4         | 1000         | 1001 0001 |
|           |              | 0100 1000 |

# Floating Point

How does RISC-V support numbers with fractions?

=> Using IEEE-754 floating point representation

# Scientific Notation is just a way to represent very large or very small numbers.

$$\Rightarrow 4500000 = \underbrace{4.5}_{\text{Coefficient}} \times \underbrace{10^6}_{\text{Base}} \quad \text{Exponent}$$
$$\Rightarrow 0.00453 = \underbrace{4.53}_{\text{Coefficient}} \times \underbrace{10^{-3}}_{\text{Base}} \quad \text{Exponent}$$

Decimal

$$\left[ \begin{array}{l} \Rightarrow 5.64 \times 10^{33} \\ \Rightarrow -2.34 \times 10^{56} \end{array} \right\} \rightarrow \text{Normalized.}$$
$$\left[ \begin{array}{l} \Rightarrow 109.64 \times 10^{33} \\ \Rightarrow 0.002 \times 10^{-4} \\ \Rightarrow +987.02 \times 10^9 \end{array} \right\} \rightarrow \text{Not Normalized.}$$

In Binary,

$$\pm 1.xxxxx_2 \times 2^{yyy}$$

IEEE-754 floating point representation:

i. Single Precision. (32 bits)

ii. Double Precision. (64 bits)

Using double precision, you can represent a larger or a smaller number than single precision.

# Normalized Number

Binary Point (representing Binary Numbers)

⇒ A binary number is Normalized if :

i) Only one digit before the binary point.

ii) And that digit must be a non-zero number.

⇒  $11.00101 \times 2^{35}$  ✗ not a normalized number.

⇒  $1.100101 \times 2^{37}$  ✓ a normalized number.

\* To normalize a number you need to shift the binary point (.) left or right until you have a single non-zero digit before the binary point.

⇒ If you shift left, the number of times you left shifted will be added with the exponent.

$$\Rightarrow 110.111 \times 2^{35}$$

$$\Rightarrow 1.10111 \times 2^{35+2}$$

⇒ If you shift right, the number of times you right shifted will be subtracted from the exponent.

$$\Rightarrow 0.00110$$

$$\Rightarrow 0.00110 \times 2^0$$

$$\Rightarrow 1.10 \times 2^{0-3} = 1.10 \times 2^{-3}$$

# IEEE-754 Floating Point Representation

|           | Sign Bit | Exponent | Fraction |
|-----------|----------|----------|----------|
| Single P. | 1 bit    | 8 bits   | 23 bits  |
| Double P. | 1 "      | 11 "     | 52 "     |

## IEEE-754 Single Precision Format (32-bit)

| (Biased) |          |          |
|----------|----------|----------|
| Sign Bit | Exponent | Fraction |
| 1        | 8        | 23       |

Sign Bit = 0  $\Rightarrow$  positive number

1  $\Rightarrow$  negative

Exponent = It will be represented as unsigned number.

8 bit unsigned binary range = 0 to  $2^8 - 1$   
= 0 to 255

But, 0000 0000 and 1111 1111 are reserved, so the range for

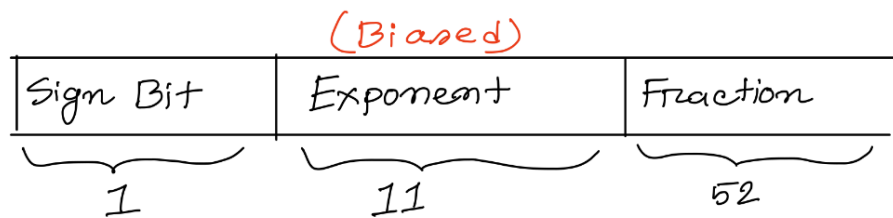
biased exponent is 1 to 254

If the size of biased exponent field is  $n$  bits, Bias =  $2^{(n-1)} - 1$

Hence, for 8 bit biased exponent, bias =  $2^7 - 1 = 127$

normalized  
 $1.1011 \times 2^{34}$   
 $1.1011 \times 2^{-8}$   
 Exponent

## IEEE-754 Double Precision Format (64-bit)



Sign Bit = 0  $\Rightarrow$  positive number

1  $\Rightarrow$  negative

Exponent = It will be represented as unsigned number.

11 bit unsigned binary range = 0 to  $2^{11}-1$   
= 0 to 2047

But, 000 0000 0000 and 111 1111 1111 are reserved, so the range for biased exponent is 1 to 2046

If the size of biased exponent field is  $n$  bits, Bias =  $2^{(n-1)} - 1$

Hence, for 11 bit biased exponent, bias =  $2^{10} - 1 = 1023$

# Decimal to IEEE-754 Floating Point Conversion

Steps:

1. Convert the decimal number to binary number.
2. Normalize the binary number.
3. Find the biased exponent.
4. Sign Bit.
5. Find the fraction.
6. Encode accordingly.

Example:

Convert 50.6749 to IEEE-754 single precision floating point representation.

Show your final answer in Hex format. Consider 10 bits while converting from decimal to binary.

Solution:

$$\begin{aligned}
 50 &= 11 \ 0010 & \cdot 6749 \times 2 &= \underline{1.3498} \\
 \cdot 6749 &= 10 \ 1011 \ 0011 \dots & \cdot 3498 \times 2 &= \underline{0.6996} \\
 & & \cdot 6996 \times 2 &= \underline{1.3992} \\
 & & \cdot 3992 \times 2 &= \underline{0.7984} \\
 & & \cdot 7984 \times 2 &= \underline{1.5968} \\
 & & \cdot 5968 \times 2 &= \underline{1.1936} \\
 & & \cdot 1936 \times 2 &= \underline{0.3872} \\
 & & \cdot 3872 \times 2 &= \underline{0.7744} \\
 & & \cdot 7744 \times 2 &= \underline{1.5488} \\
 & & \cdot 5488 \times 2 &= \underline{1.0976} \\
 & & & \vdots \\
 & & & \vdots \\
 & & & \vdots \\
 & & & \vdots \\
 & & & \vdots
 \end{aligned}$$

(i)  $50.6749 = (11 \ 0010 \cdot 10 \ 1011 \ 0011 \dots)$

$= 11 \ 0010 \cdot 10 \ 1011 \ 0011 \dots \times 2^0$

(ii)  $= 1.1001 \ 0101 \ 0110 \ 011\dots \times 2^5 \leftarrow \text{actual exponent}$

(iii)  $\text{Bias} = 2^{8-1} = 2^7 = 127$

$\therefore \text{Biased exponent} = 5 + 127 = 132 = \underline{1000 \ 0100}$

(iv) positive number; sign bit = 0

(v)  $1.1001 \ 0101 \ 0110 \ 011\dots \times 2^5$

fraction

Fraction = 1001 0101 0110 011 00000000  
 rest of the bits will be filled up by 0s.

| (Biased) |           |                             |
|----------|-----------|-----------------------------|
| Sign Bit | Exponent  | Fraction                    |
| 0        | 1000 0100 | 1001 0101 0110 011 00000000 |

$$50.6749 = 0100 \ 0010 \ 0100 \ 1010 \ 1011 \ 0011 \ 0000 \ 0000$$

$$= 0x424AB300$$

### Example:

Convert -0.0232 to 12-bit IEEE-754 representation where the size of the exponent field is 4 bits. Show your final answer in Hex format.

### Solution:

$$(i) -0.0232 = -0.0000010$$

$$(ii) -0.0000010 = 1.0 \times 2^{-6} \quad \begin{array}{l} \text{fraction} \\ \text{actual} \\ \text{exponent} \end{array}$$

$$(iii) \text{Bias} = 2^{4-1} - 1 = 7$$

$$\therefore \text{Biased Exponent} = -6 + 7 = 1 = 0001$$

$$(iv) \text{Sign Bit} = 1.$$

$$(v) \text{Fraction} = 000\ 0000$$

|   |      |          |
|---|------|----------|
| 1 | 0001 | 000 0000 |
|---|------|----------|

$$-0.000232 = 1000\ 1000\ 0000$$

$$= 0x880 \quad (\text{Ans})$$

# IEEE-754 Floating Point to Decimal Conversion

Steps:

1. Convert the Hex/Decimal number to binary number.
2. Arrange the binary number according to the given IEEE format.
3. Determine the sign.
4. Find out the actual exponent from the biased exponent field.
5. Convert Fraction to Decimal.

$$6. \text{ Final number} = (-1)^{\text{Sign Bit}} \times (1 + \text{Fraction}) \times 2^{\text{Actual Exponent}}.$$

Example:

Convert the given IEEE-754 single precision floating point number 0xF2400120 to decimal.

Solution: (i) 1111 0010 0100 0000 0000 0001 0010 0000

(ii)  $\begin{array}{c} \text{1} \\ \uparrow \\ \text{Sign} \\ \text{Bit} \end{array} \quad \begin{array}{c} \text{111 00100} \\ \uparrow \\ \text{Biased} \\ \text{Exp.} \end{array} \quad \begin{array}{c} \text{100 0000 0000 0001 0010 0000} \\ \text{Fraction} \end{array}$

(iii) sign = -

(iv) Biased exp. = 111 0010 0 = 228

$$\text{Bias} = 2^{8-1} - 1 = 127$$

$$\therefore \text{Exponent} = 228 - 127 = 101$$

(v) Fraction = 100 0000 0000 0001 0010 0000

$$= 0.100\ 0000\ 0000\ 0001\ 0010\ 0000$$

$$= 0.5000343323$$

$$(vi) \text{ Decimal Value} = (-1)^1 \times (1 + 0.5000343323) \times 2^{101}$$

$$= -1.5000343323 \times 2^{101}$$

# Floating Point Addition/Subtraction

Given A and B are both floating-point numbers.

Steps:

1. Make sure both numbers are in Binary.
2. Normalize both A and B.
3. Align the binary point so that the lower exponent matches with the higher exponent.
4. Now add or sub accordingly.
5. Normalize the result.

|                                                                                          |                                        |
|------------------------------------------------------------------------------------------|----------------------------------------|
| <u>Ex:</u> $0.999 \times 10^1 + 1.610 \times 10^{-1}$ ; size of exponent field is 3 bits |                                        |
| $= 99.99 + 0.1610$                                                                       |                                        |
| $= 1100011.111110101 + 0.0010100100$                                                     | Bias $= 2^{3-1} = 3$                   |
| $= 1.10001111110101 \times 2^6 + 1.0100100 \times 2^{-3}$                                | Biased Exp. $= 3 + 6 = 9$              |
| $= 1.10001111110101 \times 2^6 + 0.0000000010100100 \times 2^6$                          | Range $= 0 \text{ to } 2^3 - 1$        |
| $= 1.10010 \times 2^6 \text{ (Ans)}$                                                     | $= 0 \text{ to } 7$                    |
|                                                                                          | $= 1 \text{ to } 6 \text{ [reserved]}$ |
| # $110100.111011 \times 2^8 + 10110.11111 \times 2^7$                                    | $\nearrow 0 \text{ and } 7$            |
| $= 1.10100111011 \times 2^{13} + 1.011011111 \times 2^{11}$                              | upper range                            |
| $= 1.10100111011 \times 2^{13} + 0.0101101111 \times 2^{13}$                             | $9 > 6 \Rightarrow \text{So,}$         |
| $= 10.00000011010 \times 2^{13}$                                                         | overflow                               |
| $= 1.000000011010 \times 2^{14} \text{ (Ans)}$                                           |                                        |

# Floating Point Multiplication

Given A and B are both floating-point numbers.

Steps:

1. Make sure both numbers are in Binary.
2. Normalize both A and B.
3. Add the exponents.
4. Now multiply accordingly.
5. Normalize the result.
6. Determine the sign of the operation.

---

$$\underline{\text{Ex:}} \quad 1.110 \times 2^5 \times 1.11 \times 2^{-5}$$

---

$$= 1.110 \times 1.11 \times 2^{5+(-5)}$$

---

$$= 11.0001 \times 2^0$$

---

$$= 1.10001 \times 2^1 \text{ (Ans.)}$$

---

# Overflow-Underflow detection for IEEE-754 format

→ Given number is too small to represent using the mentioned system.  
Overflow / Underflow detection:

Step 1: Find the biased exponent of the answer.

Step 2:     "     "     range of the biased exponent of the given system.     (1 to upper Range)

Step 3: Detection:

if (Biased exponent  $< 1$ ):

underflow

else if (Biased exponent  $>$  upper Range)

overflow

else :  $[1 \leq \text{Biased Exp} \leq \text{upper Range}]$

No over/under flow

## Floating Point instructions in RISC-V

# Suppose, two single prec. floating point numbers  $A, B$  are stored in memory. The memory locations are directly stored in registers  $X_{10}, X_{11}$ .

Write necessary code to store the result of  $A+B$  in the memory address that is stored in  $X_{13}$ .

Sol<sup>n</sup>:

$fload \quad f_1, \text{O}(X_{10}) ; f_1 = A$

$fload \quad f_2, \text{O}(X_{11}) ; f_2 = B$

$fadd.s \quad f_3, f_1, f_2 ; f_3 = A+B$

$fsw \quad f_3, \text{O}(X_{13})$

## Floating point instruction list:

| Instruction     | Syntax & Example                                                                                          | Operation                                                                                                      | Note                                                                                                                                                                                                                              |
|-----------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>fadd.s</b>   | fadd.s rd, rs1, rs2<br>Ex: fadd.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                        | Perform single-precision floating-point addition<br>$rd = rs1 + rs2$                                           | For all the <b>red</b> marked instructions, if the numbers are in double precision then just add “d” instead of “s”<br><br>Ex:<br>fadd.d rd, rs1, rs2<br><br>Perform double-precision floating-point addition<br>$rd = rs1 + rs2$ |
| <b>fsub.s</b>   | fsub.s rd, rs1, rs2<br>Ex: fsub.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                        | Perform single-precision floating-point subtraction<br>$rd = rs1 - rs2$                                        |                                                                                                                                                                                                                                   |
| <b>fmul.s</b>   | fmul.s rd,rs1,rs2<br>Ex: fmul.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                          | Perform single-precision floating-point multiplication.<br>$rd = rs1 * rs2$                                    |                                                                                                                                                                                                                                   |
| <b>fdiv.s</b>   | fdiv.s rd,rs1,rs2<br>Ex: fdiv.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                          | Perform single-precision floating-point division.<br>$rd = rs1 / rs2$                                          |                                                                                                                                                                                                                                   |
| <b>fmadd.s</b>  | fmadd.s rd,rs1,rs2,rs3<br>Ex: fmadd.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub> , f <sub>6</sub>   | Perform single-precision fused multiply addition.<br>$rd = (rs1 * rs2) + rs3$                                  |                                                                                                                                                                                                                                   |
| <b>fnmadd.s</b> | fnmadd.s rd,rs1,rs2,rs3<br>Ex: fnmadd.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub> , f <sub>6</sub> | Perform negated single-precision fused multiply addition.<br>$rd = -(rs1 * rs2) - rs3$                         |                                                                                                                                                                                                                                   |
| <b>fmsub.s</b>  | fmsub.s rd,rs1,rs2,rs3<br>Ex: fmsub.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub> , f <sub>6</sub>   | Perform single-precision fused multiply subtraction.<br>$rd = (rs1 * rs2) - rs3$                               |                                                                                                                                                                                                                                   |
| <b>fnmsub.s</b> | fnmsub.s rd,rs1,rs2,rs3<br>Ex: fnmsub.s f <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub> , f <sub>6</sub> | Perform negated single-precision fused multiply subtraction.<br>$rd = -(rs1 * rs2) + rs3$                      |                                                                                                                                                                                                                                   |
| <b>fsqrt.s</b>  | fsqrt.s rd,rs1<br>Ex: fsqrt.s f <sub>3</sub> , f <sub>2</sub>                                             | Perform single-precision square root.<br>$rd = \sqrt{rs1}$                                                     |                                                                                                                                                                                                                                   |
| <b>feq.s</b>    | feq.s rd,rs1,rs2<br>Ex: feq.s x <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                            | Performs a comparison between two floating point numbers.<br>If both are equal stores 1 in rd else stores 0.   |                                                                                                                                                                                                                                   |
| <b>flt.s</b>    | flt.s rd,rs1,rs2<br>Ex: flt.s x <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                            | Performs a comparison between two floating point numbers.<br>If $(rs1 < rs2)$ stores 1 in rd else stores 0.    |                                                                                                                                                                                                                                   |
| <b>fle.s</b>    | fle.s rd,rs1,rs2<br>Ex: fle.s x <sub>3</sub> , f <sub>2</sub> , f <sub>5</sub>                            | Performs a comparison between two floating point numbers.<br>If $(rs1 \leq rs2)$ stores 1 in rd else stores 0. |                                                                                                                                                                                                                                   |

| Instruction | Syntax & Example                                                    | Operation                                                                                                                                               |
|-------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| flw         | flw rd,offset(rs1)<br>Ex: flw f <sub>3</sub> , 8(x <sub>5</sub> )   | Load a single-precision floating-point value from memory into floating-point register rd.                                                               |
| fld         | fld rd,offset(rs1)<br>Ex: fld f <sub>3</sub> , 16(x <sub>5</sub> )  | Load a double-precision floating-point value from memory into floating-point register rd.                                                               |
| fsw         | fsw rs2,offset(rs1)<br>Ex: fsw f <sub>3</sub> , 8(x <sub>5</sub> )  | Store a single-precision value from floating-point register rs2 to memory.                                                                              |
| fsd         | fsd rs2,offset(rs1)<br>Ex: fsd f <sub>3</sub> , 16(x <sub>5</sub> ) | Store a double-precision value from floating-point register rs2 to memory.                                                                              |
| fcvt.s.w    | fcvt.s.w rd,rs1<br>Ex: fcvt.s.w f <sub>3</sub> , x <sub>5</sub>     | Converts a 32-bit <b>signed</b> integer, in integer register rs1 into a floating-point number in floating-point register rd.                            |
| fcvt.w.s    | fcvt.w.s rd,rs1<br>Ex: fcvt.w.s x <sub>3</sub> , f <sub>5</sub>     | Convert a floating-point number in floating-point register rs1 to a signed 32-bit in integer register rd.                                               |
| fmv.w.x     | fmv.w.x rd,rs1<br>Ex: fmv.w.x f <sub>3</sub> , x <sub>5</sub>       | Move the single-precision value encoded in IEEE 754 standard encoding from the lower 32 bits of integer register rs1 to the floating-point register rd. |
| fmv.x.w     | fmv.x.w rd,rs1<br>Ex: fmv.x.w x <sub>3</sub> , f <sub>5</sub>       | Move the single-precision value in floating-point register rs1 represented in IEEE 754 encoding to the lower 32 bits of integer register rd.            |

## Formulas

1.  $Bias = 2^{(n-1)} - 1$ ;  $n = \text{Size of the exponent field}$
2.  $Biased\ Exponent = Bias + Actual\ Exponent$   
 $Actual\ Exponent = Biased\ Exponent - Bias$
3.  $Biased\ Exponent\ Range = 0\ to\ 2^n - 1$ ; [upper & lower limit is reserved]  
 $= 1\ to\ 2^n - 2$ ; [Usable range]
4.  $Actual\ Exponent\ Range = (0 - Bias)\ to\ (2^n - 1 - Bias)$   
 $= (1 - Bias)\ to\ (2^n - 2 - Bias)$ ; [Usable range]
5.  $Decimal\ Number = (-1)^{Sign\ Bit} \times (1 + Fraction) \times 2^{Actual\ Exponent}$